COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	**CPE010 - DATA STRUCTURES AND ALGORITHM**	**SECTION:**	CPE21S4
DATE PERFORMED:	08/28/26	**DATE SUBMITTED:**	2 / 9 / 26
NAME:	Mangcucang, Rhianne L.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

Activity NO. 6.1 **Searching Techniques**

1. PROCEDURE OUTPUT

What procedures did you perform?

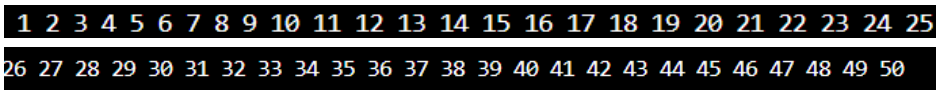
Screenshot	
Observation	The program generated 50 random numbers using 'rand' and srandtime. All numbers were properly displayed in the terminal to verify that the array was successfully generated.

Table 6-1. Data Generated and Observations.

Code

```
template <typename T>
void linearSearchArr(int maxSize, T dataFind, T data[]){
    int i = 0;
    while (i < maxSize){
        if(dataFind == data[i]){
            std::cout<<"Searching is successful\n";
            return;
        }
        i++;
    }

    std::cout<<"the search is unsuccessful\n";
}
```

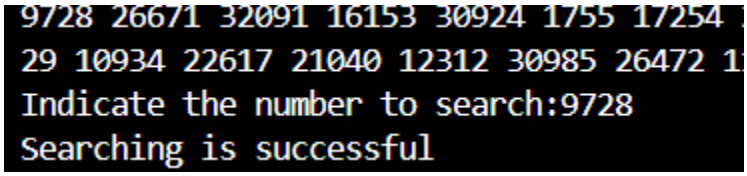
Output	
Observation	Linear search checks each element of the array one by one starting from index 0. It finds the item quickly if it is near the beginning, but it has to traverse the entire array to the end if the search value is not present.

Table 6-2a. Linear Search for Arrays

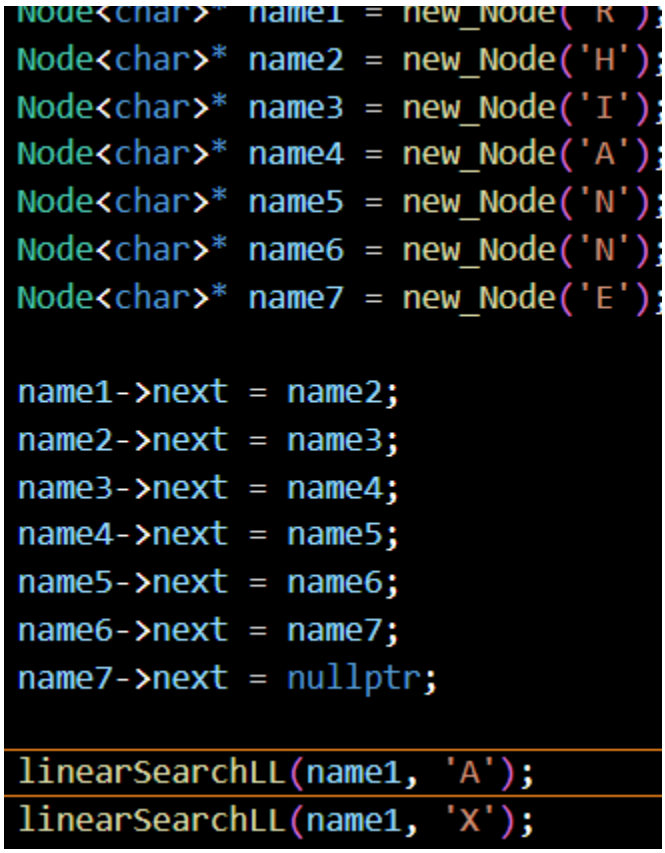
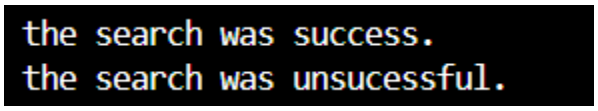
Code	
Output	
Observation	Linear search in a linked list works by moving the pointer from the head to the next node. Since linked lists do not have index numbers, it must move node by node until the target character is found or until it hits nullptr.

Table 6-2b. Linear Search for Linked List

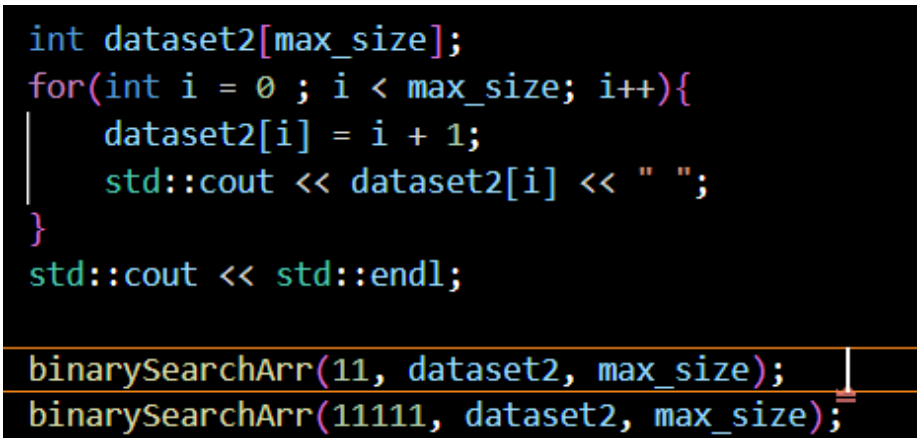
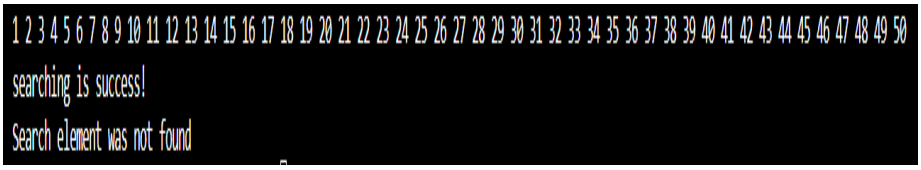
Code	
Output	
Observation	Binary search is much faster on a sorted array because it continuously divides the dataset in half using middle = low + up / 2. Direct index access allows the program to locate the middle element instantly, requiring far fewer comparisons than linear search.

Table 6-3a. Binary Search for Arrays

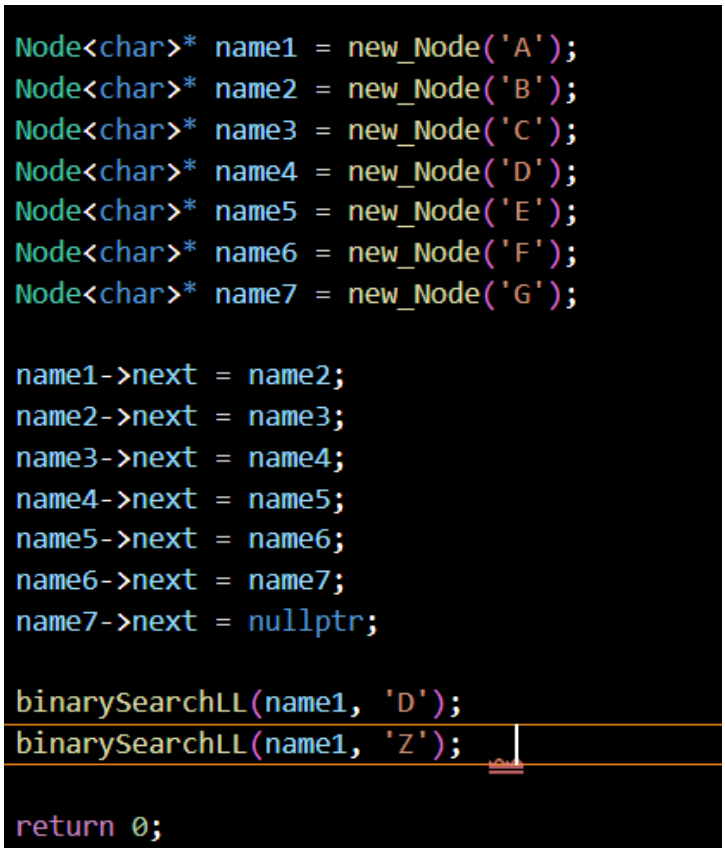
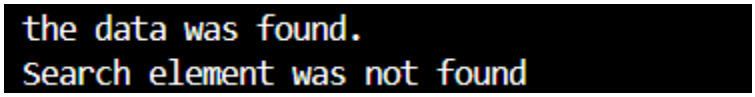
Code	
Output	
Observation	Since linked lists do not support direct indexing, the get Middle function using slow and fast pointers is used to find the center node before running binary search. It takes more time than array binary search because the pointers still need to traverse nodes just to locate the midpoint.

Table 6-3b. Binary Search for Linked List

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

CODE:

```cpp
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
};

void searchArray() {
    int arr[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
    int target = 18;
    int count = 0;

    for (int i = 0; i < 10; i++) {
        count++;
        if (arr[i] == target) {
            cout << "Array comparisons: " << count << "\n";
            break;
        }
    }
}

void searchList() {
    int arr[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};

    Node* head = new Node{arr[0], nullptr};
    Node* curr = head;
    for (int i = 1; i < 10; i++) {
        curr->next = new Node{arr[i], nullptr};
        curr = curr->next;
    }

    int target = 18;
    int count = 0;
    Node* temp = head;
```

```
37      while (temp != nullptr) {
38          count++;
39          if (temp->data == target) {
40              cout << "Linked List comparisons: " << count << "\n";
41              break;
42          }
43          temp = temp->next;
44      }
45  }
46
47  void countElements() {
48      int arr[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
49      int target = 18;
50      int count = 0;
51
52      for (int i = 0; i < 10; i++) {
53          if (arr[i] == target) {
54              count++;
55          }
56      }
57      cout << "Count: " << count << "\n";
58  }
59
60  void binaryIterative() {
61      int arr[] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
62      int target = 8;
63      int low = 0;
64      int high = 9;
65
66      while (low <= high) {
67          int mid = low + (high - low) / 2;
68          if (arr[mid] == target) {
69              cout << "Iterative Index: " << mid << "\n";
70              break;
```

```
71          } else if (arr[mid] < target) {
72              low = mid + 1;
73          } else {
74              high = mid - 1;
75          }
76      }
77  }
78
79  int binaryRecursive(int arr[], int low, int high, int target) {
80      if (low > high) return -1;
81
82      int mid = low + (high - low) / 2;
83      if (arr[mid] == target) return mid;
84      if (arr[mid] > target) return binaryRecursive(arr, low, mid - 1, target);
85      return binaryRecursive(arr, mid + 1, high, target);
86  }
87
88  void runBinaryRecursive() {
89      int arr[] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
90      int target = 8;
91
92      int index = binaryRecursive(arr, 0, 9, target);
93      cout << "Recursive Index: " << index << "\n";
94  }
95
96  int main() {
97      searchArray();
98      searchList();
99      countElements();
100     binaryIterative();
101     runBinaryRecursive();
102
```

OUTPUT:

```
Array comparisons: 2
Linked List comparisons: 2
Count: 2
Iterative Index: 3
Recursive Index: 3
```

CONCLUSION:

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use:

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.